

Adaptive Circle Clustering Algorithm: A Modified Approach from K-Means

Adriana Vento Conesa

Departamento de Ciencias de la Computación e Inteligencia Artificial

Universidad de Sevilla

Seville, Spain

Abstract—Clustering algorithms are pivotal in data analysis and pattern recognition. In this paper, we introduce an approach to clustering based on adaptive circle modeling, rooted in the k-means algorithm. Our goal is to identify the optimal set of “rings” that best model the underlying data distribution, effectively accommodating noise inherent in the dataset. Through experimentation on synthetic data, we demonstrate the effectiveness of our Adaptive Circle Clustering Algorithm in achieving accurate clustering results. It is important to note, however, that the synthetic nature of the data necessitates caution in extrapolating these findings to real-world scenarios. Nonetheless, our method holds promise for various domains, including pattern recognition and anomaly detection.

Index Terms—Clustering algorithm, Adaptive circle modeling, Pattern recognition, Noise tolerance, Measurement errors, Anomaly detection, K-Means

I. INTRODUCTION

In the realm of data analysis and pattern recognition, the k-means algorithm stands as a fundamental tool for clustering data points into distinct groups based on their similarities. K-means has found widespread application in various domains, including medicine, where it is utilized for tasks such as disease classification and pattern discoveries based on clinical data [2].

In this paper, we apply the principles of the k-means algorithm to address the challenge of modeling a collection of points within a given range, with the objective of identifying the optimal configuration of “rings” that best represent the structure of the data points. Specifically, we propose an adapted clustering algorithm that searches for circumferences, taking uncertainty explicitly into account.

Consider the scenario of a dataset comprising points scattered within a defined 2D region. The task at hand is to partition these points into clusters, or “rings” in this context, such that each point belongs to the cluster with the nearest centroid. In real-world datasets, it’s common to encounter noise and uncertainty, manifesting as outlier points or measurement errors. Therefore, our objective is not to achieve a perfect fit between the data and the modeled rings, but rather to minimize the error. Nonetheless, caution is warranted when extrapolating this application to real-world data, given that the testing phases were conducted with synthetic (albeit noisy) data. To this end, each cluster will be represented by a circumference, providing its center and radius, while each point can be associated with several (or all) clusters, featuring different “membership

degrees” that quantify the extent of their association with each cluster.

To address this challenge, we propose an adaptation of the k-means algorithm specifically tailored for ring-based clustering. Our approach involves iteratively optimizing the centroids and radii of the rings to minimize the distances between each point and its respective assigned centroid(s), a point belonging to more than one centroid if appropriate. Furthermore, our algorithm involves several noise-reduction techniques that are not typical of the traditional k-means algorithm.

Prior research has highlighted the challenges in enhancing the effectiveness of k-means, revealing that techniques such as fine-tuning centroids and increasing iterations exhibit context-dependent efficacy [4]. Additionally, larger dataset sizes seem to reduce its effectiveness [1]. While our proposed solution encompasses centroid initialization algorithms, we must acknowledge the inherent susceptibility to error in these methods, which will be further examined in this paper.

II. WORK STRUCTURE

The structure of the project files are as follows. The root, which contains:

- **clustering**: This folder contains the main modules relevant to the execution of the algorithm.
 - **algorithm**: Comprises the methods for overseeing the execution of the overall algorithm, bringing together various components to create the final executable algorithm.
 - **circlefit**: Contains the main files for the circle fitting algorithm.
 - **datamodel**: This folder contains files pertinent to the creation of the datamodel.
 - **membership**: Contains methods regarding the computation of the membership degrees for each cluster.
- **tests**: This folder contains modules relevant to the testing phases of the project.
 - **formal_tests**: Contains formal tests for clustering algorithms. These tests are relevant for the experimentation phase.
 - **informal_tests**: Contains informal tests used during the development of the algorithm. These were not used as formal test cases.

- **utils:** A folder containing all auxiliary methods used that were not directly pertinent to the main topic. It primarily comprises parsing and plotting methods.

To replicate the experimental results presented in this study, a test suite has been assembled in a single file named 'all.py', located within the 'formal_tests' folder. Detailed instructions for running the test suite are provided in the accompanying documentation.

III. METHODOLOGY AND APPROACH

A. Data Structure Design

The problem is modeled through the design of a Cluster class. This class represents the center of the final circumference, its radius, and the membership degrees. With this structure, the center and the radius is updated in each iteration of the algorithm. The membership degrees are stored as a list, where each index corresponds to a point in the dataset, and the value at that index represents the degree of membership of that point to the cluster. This means that every point in the dataset has an associated membership degree for each cluster, regardless of the number of clusters present in the problem.

We considered using a dictionary as an alternative, but due to the requirement of having a membership degree per point, the added complexity wasn't justified. Using a list for membership degrees, where each index corresponds to a point, proved to be a simpler and more straightforward approach for our problem.

B. Estimation of Circular Trajectory Parameters

For this section, we must consider three different design phases:

- **Single circumference estimation:** In which it is assumed that there is only circumference circle in the dataset, and the parameters are estimated accordingly.
- **Single arc estimation:** In which it is no longer assumed that there is a circumference circle in the dataset, now accounting for the potential presence of (one) circular sector as well. However, this consideration was omitted from the global algorithm.
- **Multiple circumferences estimation:** In which it is no longer assumed that there is only one circumference in the dataset, and the degree to which the points belong to the circumferences are taken into account when estimating the parameters.

The third phase of the algorithm depended on the design decisions made in the first phase. Therefore, let's begin by explaining the first phase.

The algorithm relied on the mathematical concept of the barycenter, which is equivalent to the centroid or center of the cluster. In this computation, all points in the dataset were considered to determine the barycenter. For the radius, we adopted a strategy dependent on the barycenter, serving as a point of reference between the dataset points and our initial position. To achieve this, we calculated the average distance between a set of points and the barycenter. For each point,

we calculated the distance to the center using the Euclidean formula (1).

$$\text{Euclidean Distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (1)$$

These steps are semi-standard procedures, as they are well-known components of a typical k-means implementation process [5]. However, phase three required further adaptations in order to properly accommodate our datasets.

Algorithm 1: Fitting Circles with Weighted Points

Require: *points*: List of points in the dataset
Require: *clusters*: List of cluster objects
Ensure: *centers*: List of circumference centers
Ensure: *radii*: List of circumference radii
Ensure: *assigned_points*: List of points assigned to each cluster

```

1: assigned_points  $\leftarrow$  []
2: centers  $\leftarrow$  []
3: radii  $\leftarrow$  []
4: cluster_points  $\leftarrow$  List of empty lists, one for each cluster
5: for point in points do
6:   Find the cluster with maximum membership degree for point and assign it to max_membership_index
7:   Add point to cluster_points[max_membership_index]
8: end for
9: equal_degrees  $\leftarrow$  []
10: for point in points do
11:   degrees  $\leftarrow$  List of membership degrees for each cluster for point
12:   if degrees are approximately equal then
13:     Add the index of point to equal_degrees
14:   end if
15: end for
16: if equal_degrees is not empty then
17:   for index in equal_degrees do
18:     for cluster in cluster_points do
19:       if the point at index is in cluster then
20:         Add the point at index to cluster
21:       end if
22:     end for
23:   end for
24: end if
25: for cluster in clusters do
26:   center  $\leftarrow$  Calculate the barycenter of cluster with cluster_points
27:   radius  $\leftarrow$  Calculate the radius of cluster from center and cluster_points
28:   Add cluster's points, center, and radius to respective lists
29: end for
30: return centers, radii, assigned_points

```

In phase three, the computation of the center and radius of the circumference remained unchanged. However, the inputs to the functions were modified according to the scheme presented in Algorithm 1.

A notable consideration in designing this algorithm is the decision to include the same point in different clusters if the distances from the point to each cluster fall within the threshold established by the algorithm. This approach enables the algorithm to consider points that lie between clusters, allowing points to belong to multiple clusters if necessary. Otherwise, assigning points to each cluster remains straightforward. The process involves taking the membership degrees for each point, computing the maximum value among them, and then assigning the point to the cluster with the highest degree value.

Additionally, phase two of the design process aimed to approximate a circumference based on partial sector data. Despite its potential value, this phase was not integrated into the final algorithm but was left as a potential path for future enhancements. Nevertheless, for the sake of completeness, we must comment on it. The process starts with an initial guess for the center of the circle, which is calculated as the mean of the arc points, and an initial guess for the radius, which is set as the maximum distance between the arc points and the guessed center. Then, we perform an optimization process which aims to refine the initial guesses by minimizing the sum of distances between the arc points and the circle. This is done using an optimization algorithm called the Nelder-Mead method [7], implemented in the minimize function from *scipy.optimize* [8]. The optimization algorithm iteratively adjusts the center coordinates and radius of the circle to minimize the objective function. We can observe an application of the method in Figure 1.

C. Calculation of Membership Degrees

As noted in earlier sections, the membership degrees comprise a list for each cluster, containing degrees for all points in the dataset. Each index within the list corresponds to the position of the respective point in the dataset. However, the implementation of the calculation of membership degrees follows a different approach.

Instead, we introduce a point and a list of clusters. For each cluster, we compute its degree with respect to the chosen point. To achieve this, we calculate the distance between a point and a circle defined by the cluster's center and radius (2). The distance was once again computed using the Euclidean distance formula (1).

$$d(P, c) = |d(P, O) - r| \quad (2)$$

We then normalized the degrees to ensure that they sum to one.

D. Implementation of the Main Algorithm

This section compiles all necessary methods for the implementation of the algorithm into a single executable function.

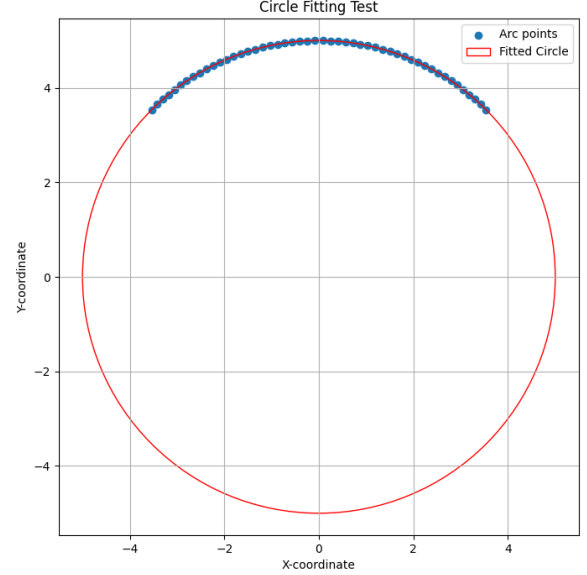


Fig. 1. Arc parameter estimation example.

Therefore, it is divided into several sections, each of which we will explore in detail.

1) *Cluster Initialization*: This method generates the initial centers of the clusters for the algorithm execution. For this purpose, the K-Means++ algorithm [6] was implemented, which is the most common initialization approach after random initialization [4]. A random initialization method was included for testing purposes.

2) *Circumference Parameters Update*: This method applies Algorithm 1 to update the centers and radii of the different clusters in the problem.

3) *Membership Degrees Update*: Similarly, this method applies the calculation of the membership degrees as explained in Section III-C.

4) *Ring Clustering*: This method consolidates all the methods explained in this section. A detailed procedure can be found in Algorithm 2. The only significant design decision was to combine a maximum number of iterations with a convergence checker, rather than implementing a strict iteration limit alone. To achieve this, we compute the distance between the centers obtained in the previous iteration and the new centers. If the distance falls below the tolerance level, the algorithm stops, reducing the total number of iterations needed to obtain a result. A separate ring clustering method with a random initialization method was included for testing purposes.

E. Discarded Approaches

Although every project undergoes a phase of trial-and-error, the initial proposal for the circumference fitting algorithm aimed to be more innovative than its current state (see Algorithm 1). There was an attempt to estimate the

Algorithm 2: Ring Clustering

Require: *points*: List of points
Require: *num_clusters*: Number of clusters
Require: *max_iterations*: Maximum number of iterations (default: 2500)
Require: *tol*: Tolerance for convergence (default: $1e - 8$)
Ensure: *clusters*: List of clusters
Ensure: *assigned_points*: List of points assigned to clusters

- 1: Initialize clusters with *num_clusters* and *points*
- 2: *prev_centers* $\leftarrow$ Initial centers of each cluster
- 3: **for** *_* **in** *range*(*max_iterations*) **do**
- 4: Update membership degrees for *points* in *clusters*
- 5: *assigned_points* $\leftarrow$ Update clusters with *points* and *clusters*
- 6: *new_centers* $\leftarrow$ Store new centers for each cluster
- 7: Calculate maximum *center_movement* using Euclidean distance between *new_centers* and *prev_centers*
- 8: **if** maximum *center_movement* is less than *tol* **then**
- 9: **break**
- 10: **end if**
- 11: *prev_centers* $\leftarrow$ *new_centers*
- 12: **end for**
- 13: **return** *clusters*, *assigned_points*

parameters of the circumference using the "Least Squares Fitting" algorithm for circles [3]. While the implementation for the single circumference method described in Section III-B yielded accurate results, its adaptation to a weighted algorithm proved challenging. Due to time constraints, this aspect wasn't further evaluated. However, we believe that it warrants further investigation, as its initial evaluation showed to be a better predictor than our current implementation.

IV. EXPERIMENTAL STUDY

A. Testing Methodology

The test cases to be analyzed in this section are as follows:

- 1) **Test case one:** One circumference, no noise, low noise and high noise.
- 2) **Test case two:** Two circumferences, separated, no noise, low noise and high noise.
- 3) **Test case three:** Two circumferences, overlapping, no noise, low noise and high noise.
- 4) **Test case four:** Two circumferences, concentric, no noise, low noise and high noise.
- 5) **Test case five:** Three circumferences, separated, no noise, low noise and high noise.

With no noise, denoted by a noise factor of 0.0; low noise, with a factor of 0.1; and high noise, with a factor of 0.2.

Each test case offers two modes of execution: one with randomly chosen centroid initialization, and the other using the K-Means++ algorithm, as discussed in Section III-D1.

Each mode of execution will be repeated five times for each test case and corresponding noise level, of which we have defined three different ones. In total, we will have thirty data points per test case. Consequently, the analysis of each test case will be divided into two subsequent tables, depending on the initialization method used. This analysis will cover the number of iterations, elapsed time, and error. The error will be computed as the sum of the errors accumulated by the centroids and the radii (of all clusters). The center error is computed as the Euclidean distance (1) between the estimated center and the actual center. The radius error is computed as the absolute difference between the estimated radius and the actual radius. The default tolerance and maximum iterations defined by the algorithm will be employed, as outlined in Algorithm 2.

We will classify a result as "acceptable" when the graphical representation of the estimated centers and radii fits well with the data. This can also be observed through the error parameter in the experimentation section. Generally, a result is considered unacceptable when the error parameter exceeds 1. This threshold was determined by analyzing the relationship of the visual representation with the data.

The points for these scenarios were synthetically generated using a Python script and then stored in various CSV files. Detailed instructions for creating new test cases are provided in the accompanying documentation.

B. Test Case One

The test case can be observed through its graph in Figure 2. Overall, the results obtained in Table I and Table II were accurate compared to the predicted data. As expected, the noise factor has a directly proportional impact on the amount of error, increasing as the noise factor becomes larger. However, interestingly, the error for the 0.1 noise factor is larger than that of the 0.2 factor, which is a trend not observed in the other test cases. There are no notable differences between the results of this test case when using heuristic and random initialization. Both methods yield similar and acceptable results.

TABLE I
ONE CIRCUMFERENCE - HEURISTIC INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0113	0.0115	0.0131	0.0135	0.0184
	Iterations	1	1	1	1	1
	Error	0.0764	0.0764	0.0764	0.0764	0.0764
0.1	Elapsed Time	0.0127	0.0119	0.0113	0.0114	0.0119
	Iterations	1	1	1	1	1
	Error	0.1250	0.1250	0.1250	0.1250	0.1250
0.2	Elapsed Time	0.0109	0.0139	0.0126	0.0120	0.0123
	Iterations	1	1	1	1	1
	Error	0.0836	0.0836	0.0836	0.0836	0.0836

C. Test Case Two

The test case can be observed through its graph in Figure 3. Similarly to the previous test case, Tables III and IV exhibit good adherence to the expected results. We still do not

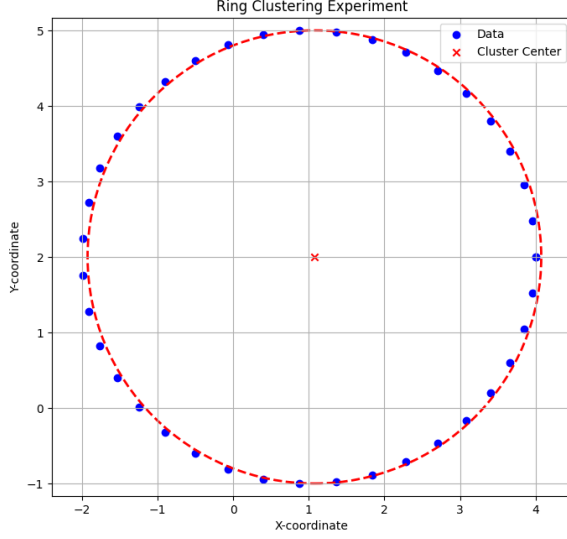


Fig. 2. Test case one evaluation: no noise setting.

TABLE II
ONE CIRCUMFERENCE - RANDOM INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0123	0.0123	0.0115	0.0123	0.0184
	Iterations	1	1	1	1	1
	Error	0.0764	0.0764	0.0764	0.0764	0.0764
0.1	Elapsed Time	0.0120	0.0114	0.0123	0.0120	0.0120
	Iterations	1	1	1	1	1
	Error	0.1250	0.1250	0.1250	0.1250	0.1250
0.2	Elapsed Time	0.0149	0.0114	0.0128	0.0119	0.0121
	Iterations	1	1	1	1	1
	Error	0.0836	0.0836	0.0836	0.0836	0.0836

observe a difference in error between heuristic and random initialization. However, there is a slight increase in the number of iterations for random initialization compared to heuristic initialization. The elapsed time remains similar in both cases. As expected, the error continues to increase with the addition of more clusters and noise.

TABLE III
TWO SEPARATED CIRCUMFERENCES - HEURISTIC INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0031	0.0019	0.0032	0.0	0.0
	Iterations	1	1	2	2	1
	Total Error	0.0636	0.0636	0.0636	0.0636	0.0636
0.1	Elapsed Time	0.0	0.0151	0.0156	0.0	0.0
	Iterations	1	1	2	1	1
	Total Error	0.0794	0.0794	0.0794	0.0794	0.0794
0.2	Elapsed Time	0.0175	0.0	0.0042	0.0155	0.0096
	Iterations	2	3	2	4	1
	Error	0.2072	0.2072	0.2072	0.2072	0.2072

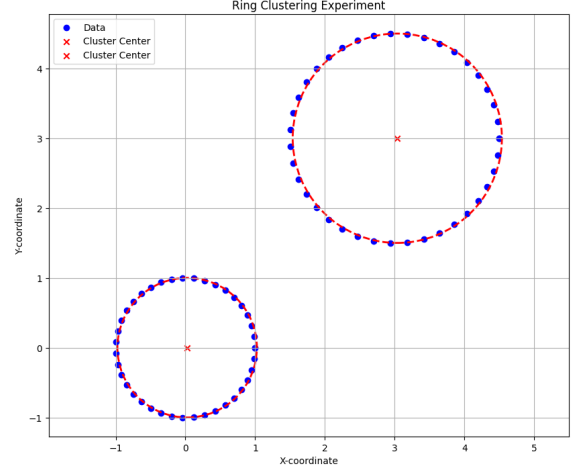


Fig. 3. Test case two evaluation: no noise setting.

TABLE IV
TWO SEPARATED CIRCUMFERENCES - RANDOM INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0229	0.0247	0.0	0.0156	0.0334
	Iterations	3	9	3	3	11
	Total Error	0.0636	0.0636	0.0636	0.0636	0.0636
0.1	Elapsed Time	0.0	0.0350	0.0	0.0086	0.0165
	Iterations	2	7	3	2	2
	Total Error	0.0794	0.0794	0.0794	0.0794	0.0794
0.2	Elapsed Time	0.0088	0.0310	0.0211	0.0157	0.01776
	Iterations	2	3	3	3	4
	Error	0.2072	0.2072	0.2072	0.2072	0.2072

D. Test Case Three

The test case can be observed through its graph in Figure 4. We are testing a more complex example, which yields significantly different results from the previous test cases. We can now indeed observe a notable difference between the test using a heuristic initialization method and the one that does not. As seen in Table V, all trials have errors within the bounds of "acceptable" results. For the first time, we observe examples of reaching the maximum iteration number predefined by the algorithm (2500), as the convergence condition was always met before these trials. However, this does not prevent us from achieving acceptable results, as the trials with that iteration number still have errors comparable to those in which the convergence condition was met.

In Table VI, we observe a different dynamic. For a noise factor of 0.0, we find that 60% of the results yield undesirable errors. The same percentage is obtained for the 0.1 noise factor, while the 0.2 factor skyrockets to an 80% failure rate. Thus, we can observe a significant positive impact of using the heuristic in this test case. As for the elapsed time, it remains mostly the same, except for cases in which we reach the maximum number of iterations, where the difference is notably significant.

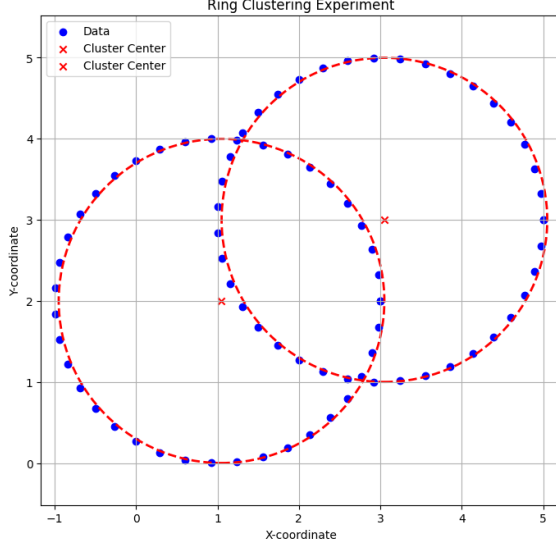


Fig. 4. Test case three evaluation: no noise setting.

TABLE V
TWO OVERLAPPING CIRCUMFERENCES - HEURISTIC INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0156	0.0	0.0120	0.0131	0.0120
	Iterations	9	6	6	6	6
	Total Error	0.1018	0.1018	0.1018	0.1018	0.1779
0.1	Elapsed Time	3.6740	3.5505	3.5365	0.01669	3.5287
	Iterations	2500	2500	2500	7	2500
	Total Error	0.0767	0.1613	0.1613	0.2115	0.1613
0.2	Elapsed Time	0.0157	0.0202	0.0172	0.0250	0.0168
	Iterations	9	10	7	10	10
	Error	0.2969	0.2969	0.2969	0.2969	0.2969

E. Test Case Four

The test case can be observed through its graph in Figure 5. This test case poses the most challenges due to the nature of the data. It involves concentric circles, which means that the algorithm must compute both clusters with the same center but different radii. This presents a significant challenge, as observed in Tables VII and VIII. This complexity likely contributes to the higher error rates and variability in results observed in this test case compared to others.

In Table VII we observe a 0% success rate for both the 0.0 and 0.2 noise factor. We obtain a single acceptable result in noise factor 0.0, resulting in a 20% success rate. Similarly, in Table VIII, we find a 0% success rate for the 0.1 and 0.2 noise factors, but a 20% success rate for the 0.0 noise factor. We could not determine any link between these two "acceptable" instances, as they have different noise factors, elapsed times, and iterations. We cannot say that the heuristic or random initialization had any effect in this test case, as there aren't any notable trends in the parameters. Overall, this test case yielded unsatisfactory results and requires further evaluation.

TABLE VI
TWO OVERLAPPING CIRCUMFERENCES - RANDOM INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0156	0.0469	0.0212	0.0337	0.0212
	Iterations	5	11	7	7	7
	Total Error	0.1779	0.1018	2.5023	2.5023	3.5645
0.1	Elapsed Time	0.0307	3.5584	0.0312	3.5889	3.5231
	Iterations	3	2500	7	2500	2500
	Total Error	2.4688	0.1613	2.4688	0.1613	0.1613
0.2	Elapsed Time	0.0312	0.0231	0.0131	0.0122	0.0236
	Iterations	10	5	5	6	4
	Error	0.2969	3.0332	2.5871	3.0332	3.6595

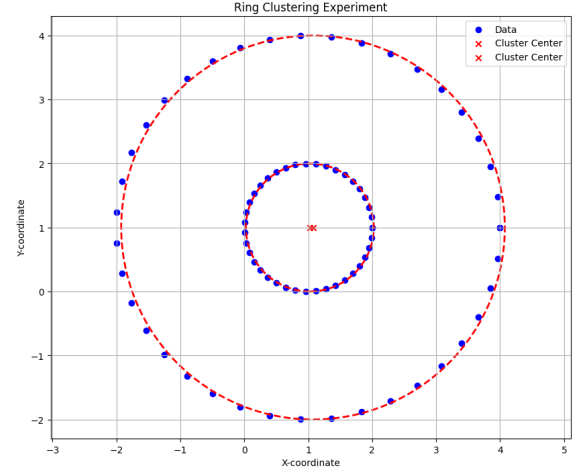


Fig. 5. Test case four evaluation: no noise setting.

F. Test Case Five

The test case can be observed through its graph in Figure 6. This case is simpler than the previous one, as it is the first and only test case in which we consider three possible clusters. It was created solely to observe if the behavior might change or fail with the addition of clusters. By observing Tables IX and X, we determine that the behavior is indeed altered by the addition of another cluster. In contrast to the separate two-circumference test case in Section IV-C, we encounter results that cannot be categorized as acceptable. Specifically, the random initialization method demonstrates a 0% success rate for this test case, while the heuristic initialization method achieves an 86% success rate. We attribute this outcome to the initialization method, as the other studied parameters remain within the expected levels.

V. CONCLUSIONS

In conclusion, our study presents an algorithmic approach to the problem of finding optimal rings to model point cloud data within a given range. Through experimentation, we have observed that our algorithm performs well for individual cases and simple scenarios involving two circumferences, regardless of the initialization method used, whether it be K-Means++ or

TABLE VII
TWO CONCENTRIC CIRCUMFERENCES - HEURISTIC INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0087	0.0016	5.1550	0.0120	0.0168
	Iterations	3	3	2500	3	5
	Total Error	3.1516	3.1786	2.7902	3.1792	2.9274
0.1	Elapsed Time	0.0069	0.0156	0.0159	0.0330	0.0156
	Iterations	7	4	3	10	6
	Total Error	2.9266	3.1896	3.1918	0.1586	2.8684
0.2	Elapsed Time	0.0089	0.0101	0.0201	0.0156	0.0308
	Iterations	3	3	8	3	7
	Error	3.0639	3.0639	2.8822	3.0639	2.7698

TABLE VIII
TWO CONCENTRIC CIRCUMFERENCES - RANDOM INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0	0.0299	0.0348	0.0130	0.0331
	Iterations	3	3	12	3	3
	Total Error	3.1638	3.1689	0.1018	3.1689	3.1689
0.1	Elapsed Time	0.0307	0.0162	0.0168	0.0262	5.4504
	Iterations	3	3	3	4	2500
	Total Error	3.0733	3.1867	3.0638	3.0638	3.1739
0.2	Elapsed Time	0.0024	0.0209	0.0389	0.0161	0.0196
	Iterations	3	9	5	9	9
	Error	3.1224	2.8822	3.1238	2.8822	2.8822

random initialization. However, when dealing with more complex cases such as overlapping circumferences or scenarios with more than two circumferences, our results indicate that employing a heuristic for initialization significantly improves the success rate.

Interestingly, for cases where the circumferences are concentric, our algorithm exhibited poor performance, and the initialization method did not offer significant improvement. This limitation may be attributed to the circle fitting technique we employed, which computes the circle parameters based on the barycenter and average distances to the points. Therefore, we propose future research avenues to explore alternative

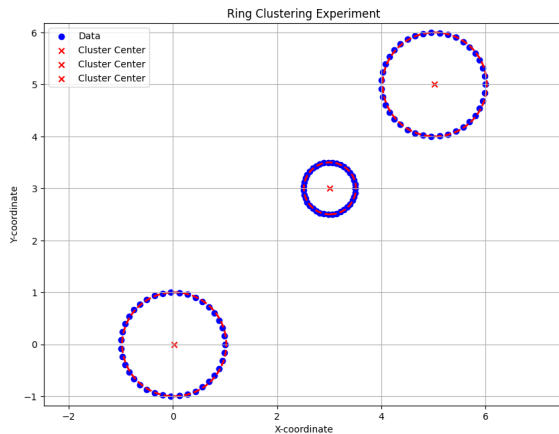


Fig. 6. Test case five evaluation: no noise setting.

TABLE IX
THREE SEPARATED CIRCUMFERENCES - HEURISTIC INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0156	0.0	0.0168	0.0156	0.0070
	Iterations	1	1	8	1	2
	Total Error	0.0636	3.6604	0.0636	0.0636	0.0636
0.1	Elapsed Time	0.0	0.0	0.0	0.0	0.0069
	Iterations	2	1	1	2	3
	Total Error	0.1294	0.1294	3.6787	0.1294	0.1294
0.2	Elapsed Time	0.0051	0.0	0.0	0.0167	0.0
	Iterations	3	2	1	5	2
	Error	0.2242	0.2242	0.2242	0.2242	0.2242

TABLE X
THREE SEPARATED CIRCUMFERENCES - RANDOM INITIALIZATION

Noise	Parameters	T1	T2	T3	T4	T5
0.0	Elapsed Time	0.0313	0.0502	0.0670	0.0312	0.0240
	Iterations	2	5	5	2	2
	Total Error	3.5165	3.4577	3.2840	3.4258	3.4040
0.1	Elapsed Time	0.0476	0.0308	0.0456	0.0156	0.0206
	Iterations	2	3	3	2	4
	Total Error	3.5894	3.2828	3.2825	3.5495	3.2714
0.2	Elapsed Time	0.0616	0.0504	0.0630	0.0333	0.0667
	Iterations	17	3	6	3	17
	Error	3.5315	3.3785	3.4202	3.5315	3.5315

circle fitting methods, potentially incorporating said estimation for arcs as well, making use of what was introduced in Section III-B or exploring discarded ideas proposed in Section III-E.

Moreover, our study highlights the need for robust noise handling techniques, as certain types of noise, such as outliers, still impact our results. Investigating strategies to handle noise more effectively could enhance the robustness of our algorithm in real-world applications.

In summary, while our algorithm shows promise in certain scenarios, there are clear opportunities for further refinement and enhancement.

REFERENCES

- [1] Ashabi, A., Sahibuddin, S. B., & Salkhordeh Haghighi, M. (2021). The Systematic Review of K-Means Clustering Algorithm. *Proceedings of the 2020 9th International Conference on Networks, Communication and Computing*, 13–18. <https://doi.org/10.1145/3447654.3447657>
- [2] An Enhanced k-Means Clustering Algorithm for Pattern Discovery in Healthcare Data—Ramzi A. Haraty, Mohamad Dimishkieh, Mehedi Masud, 2015. (n.d.). Retrieved May 27, 2024, from <https://journals.sagepub.com/doi/10.1155/2015/615740>
- [3] Chernov, N., & Lesort, C. (2005). Least Squares Fitting of Circles. *Journal of Mathematical Imaging and Vision*, 23(3), 239–252. <https://doi.org/10.1007/s10851-005-0482-8>
- [4] Fränti, P., & Sieranoja, S. (2019). How much can k-means be improved by using better initialization and repeats? *Pattern Recognition*, 93, 95–112. <https://doi.org/10.1016/j.patcog.2019.04.014>
- [5] Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media, Inc.
- [6] Arthur, David & Vassilvitskii, Sergei. (2007). K-Means++: The Advantages of Careful Seeding. *Proc. of the Annu. ACM-SIAM Symp. on Discrete Algorithms*. 8. 1027-1035. [10.1145/1283383.1283494](https://doi.org/10.1145/1283383.1283494).
- [7] Gao, F., & Han, L. (2012). Implementing the Nelder-Mead simplex algorithm with adaptive parameters. *Computational Optimization and Applications*, 51, 259–277. <https://doi.org/10.1007/s10589-010-9329-3>
- [8] SciPy documentation—SciPy v1.13.1 Manual. (n.d.). Retrieved May 31, 2024, from <https://docs.scipy.org/doc/scipy/index.html>

- [9] Bhattacharjee, A. (2023, September 24). Implementing K-Means Clustering from Scratch in Python. Medium. <https://medium.com/@avijit.bhattacharjee1996/implementing-k-means-clustering-from-scratch-in-python-a277c23563ac>
- [10] Alpaydin, E. (2020). Introduction to machine learning. MIT press.
- [11] KMeans. (n.d.). Scikit-Learn. Retrieved June 3, 2024, from <https://scikit-learn/stable/modules/generated/sklearn.cluster.KMeans.html>
- [12] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.